

Transformers pour la régression in-context, les systèmes linéaires et les problèmes inverses en EDP

Revue critique, cartographie des lacunes et formulation de la question de recherche

Note de travail pour Janis Aiad

État de la littérature au 31 juillet 2026

Résumé

Cette note cartographie la littérature reliant l'apprentissage en contexte (in-context learning, ICL), la régression linéaire, les Transformers vus comme algorithmes d'optimisation, l'apprentissage d'opérateurs et les problèmes directs ou inverses gouvernés par des équations aux dérivées partielles (EDP). Elle distingue quatre objets souvent confondus : (i) apprendre directement un surrogate de l'opérateur solution, (ii) identifier en contexte une loi ou un opérateur caché, (iii) dérouler dans les activations un solveur numérique, et (iv) apprendre un préconditionneur utilisé par un solveur externe. Le papier de Takanami, Takahashi et Kabashima sur les tâches de régression de faible rang est analysé en détail : son calcul de répliques prédit la loi du prédicteur et une transition de phase en grande dimension, mais ne constitue ni une analyse d'un solveur profond, ni un résultat d'EDP. La lacune centrale identifiée est l'absence d'un cadre qui combine identification en contexte d'un opérateur elliptique, décodeur itératif accéléré à poids partagés, préconditionnement conditionné par le prompt, décomposition complète des erreurs et théorie haute dimension du risque. Nous formulons en conséquence une question de recherche et un protocole expérimental falsifiable autour d'un décodeur HeavyBall/Chebyshev, avec PCG comme référence numérique et Richardson comme baseline mécanistique.

Table des matières

1	Périmètre, méthode et vocabulaire	3
1.1	Périmètre de la revue	3
1.2	Quatre sens différents de « Transformer pour résoudre une EDP »	3
1.3	Deux inversions à ne pas confondre	4
2	ICL général : origine et grandes familles d'explications	4
3	ICL pour la régression linéaire : fondations empiriques et algorithmiques	5
3.1	Le paradigme expérimental fondateur	5
3.2	Attention linéaire comme pas de gradient préconditionné	5
3.3	Ce que l'entraînement apprend réellement	5
3.4	Au-delà du premier ordre	5
4	Profondeur, boucles et conditionnement	6
4.1	Pourquoi une couche n'est pas une itération universelle	6
4.2	Transformers bouclés et poids partagés	6
4.3	Régression structurée, parcimonieuse et mélanges	6
5	Théorie haute dimension : matrices aléatoires et répliques	7
5.1	Limite proportionnelle de l'attention linéaire	7
5.2	Le papier de faible rang 2510.04548	7
5.3	Rigueur possible au-delà des répliques	8

6	Des systèmes linéaires aux opérateurs et aux EDP	8
6.1	ICL de systèmes linéaires	8
6.2	Operator learning classique	8
6.3	Transformers comme neural operators	8
6.4	Pré-entraînement multi-EDP et modèles de fondation	9
6.5	In-context operator learning	9
7	Problèmes inverses en EDP et architectures Transformer	9
7.1	Surrogates inverses et opérateurs inverses	9
7.2	Formulation faible et identification de faible dimension	10
8	Préconditionnement appris et solveurs hybrides	10
8.1	Pourquoi cette littérature est une baseline obligatoire	10
8.2	Trois architectures possibles pour le projet	10
8.3	HeavyBall, Chebyshev et PCG : comparaison honnête	11
9	Décomposition des erreurs : ce qu’une théorie complète doit prouver	11
10	Tableau critique de la littérature la plus proche	12
11	La lacune précise et la question que nous améliorons	12
11.1	Lacune	12
11.2	Question principale proposée	13
11.3	Sous-questions falsifiables	13
12	Contributions réalistes et ordre de travail	13
13	Protocole expérimental minimal pour soutenir les revendications	14
13.1	Axes de données	14
13.2	Métriques	14
13.3	Tests mécanistiques	15
14	Conclusion	15

1 Périmètre, méthode et vocabulaire

1.1 Périmètre de la revue

La revue couvre les travaux primaires disponibles jusqu’au 31 juillet 2026 dans les catégories suivantes :

1. ICL de fonctions linéaires, affines, parcimonieuses, de mélanges et de noyaux ;
2. Transformers comme implémentations ou approximations d’algorithmes d’optimisation et d’algèbre linéaire ;
3. profondeur, partage des poids, modèles bouclés et calcul au temps d’inférence ;
4. théorie statistique non asymptotique, limites proportionnelles, matrices aléatoires, répliques et changement de distribution ;
5. neural operators et Transformers pour EDP, y compris modèles de fondation et in-context operator learning ;
6. problèmes inverses en EDP, préconditionneurs appris, multigrille appris et couplage réseaux–Krylov.

Une exhaustivité littérale de tous les articles contenant le mot « Transformer » serait peu informative. Le critère d’inclusion retenu est la présence d’au moins un lien conceptuel direct avec l’un des objets suivants : identification d’une tâche à partir d’exemples, résolution d’un système linéaire dépendant de la tâche, apprentissage d’un opérateur entre espaces de fonctions, inversion d’un paramètre d’EDP, ou accélération d’un solveur.

1.2 Quatre sens différents de « Transformer pour résoudre une EDP »

La première clarification est taxonomique. Les quatre paradigmes ci-dessous ne répondent pas à la même question.

Paradigme	Entrée–sortie	Ce qui est appris	Garantie pertinente
Surrogate d’opérateur	coefficient/champ initial $\mapsto$ solution	une approximation globale de l’opérateur solution	approximation, généralisation, stabilité et invariance à la discrétisation
ICL d’opérateur	prompt de paires fonctionnelles + requête $\mapsto$ réponse	adaptation à une loi cachée sans mise à jour des poids	erreur en contexte, diversité des tâches, transfert ID/OOD
Transformer-solveur	matrice ou prompt + second membre $\mapsto$ itérés	un algorithme de résolution dans les activations	erreur par itération, stabilité spectrale, coût par produit matrice–vecteur
Préconditionneur appris	opérateur/géométrie $\mapsto B_\theta$, puis solveur externe	une transformation spectrale utile	SPD ou flexibilité du solveur, conditionnement effectif, temps total

DeepONet et FNO appartiennent d’abord à la première classe [38, 39, 40]. ICON appartient à la deuxième [52, 53]. Les constructions de gradient descent, Newton ou Richardson appartiennent à la

troisième [11, 19, 33]. NeuralIF et FCG-NO appartiennent à la quatrième [61, 62]. Une contribution doit annoncer explicitement sa classe, sinon une comparaison de performances ou de garanties devient trompeuse.

1.3 Deux inversions à ne pas confondre

Pour une famille elliptique

$$\mathcal{L}_{a,V}u = -\nabla \cdot (a\nabla u) + Vu = f, \quad u|_{\partial\Omega} = 0, \quad (1)$$

deux problèmes inverses distincts apparaissent :

1. *identification de loi* : retrouver a , V , ou une matrice de Galerkin A à partir de paires (f_i, u_i) ;
2. *inversion numérique* : pour un opérateur estimé $\hat{A}$ et un nouveau second membre $f_\star$, calculer $\hat{A}^{-1}f_\star$.

Le premier est statistique et potentiellement mal posé ; le second est un problème de solveur linéaire. Le projet considéré ici comporte les deux. Une borne finale doit donc séparer l’erreur d’encodeur, l’erreur de discrétisation, le biais de régularisation et l’erreur de profondeur finie du décodeur.

2 ICL général : origine et grandes familles d’explications

Brown et al. ont popularisé le comportement few-shot des grands modèles de langage : une tâche est spécifiée par des exemples placés dans le prompt, sans mise à jour des poids à l’inférence [1]. L’ICL ne se réduit cependant pas à une seule cause. La littérature générale distingue au moins quatre explications compatibles plutôt que mutuellement exclusives.

Inférence bayésienne implicite. Xie et al. modélisent le pré-entraînement comme l’apprentissage d’un mélange de concepts latents ; conditionner sur un prompt revient alors à inférer le concept courant [2]. Cette lecture est naturelle pour une famille d’EDP : le prompt donne de l’information sur l’opérateur latent. Elle ne dit pas encore comment inverser efficacement l’opérateur une fois identifié.

Mesa-optimisation. Dai et al. et von Oswald et al. interprètent la passe avant comme une procédure d’optimisation interne [3, 11]. Cette famille d’explications motive directement un décodeur itératif. Elle doit être testée contre des alternatives entrée–sortie équivalentes, car la proximité des prédictions ne suffit pas à identifier un circuit.

Induction et structure de la distribution. Olsson et al. relient l’émergence de l’ICL à des têtes d’induction capables de copier ou prolonger des motifs [4]. Chan et al. montrent que des propriétés de la distribution de séquences, notamment répétition et burstiness, favorisent l’émergence de l’ICL [5]. Un autre travail de Chan et al. distingue la généralisation issue des informations stockées dans les poids de celle issue du contexte [6]. Pour les EDP, l’analogie est la distinction entre prior de famille appris dans les poids et opérateur particulier révélé par le prompt.

Rôle sémantique des démonstrations et apprenabilité. Min et al. montrent, sur des tâches de langage, que la distribution des entrées, le format et l’espace des labels peuvent compter davantage que la correction de chaque label individuel [7]. Wies et al. étudient formellement l’apprenabilité de règles ICL par pré-entraînement [8]. Ces résultats invitent à ne pas supposer qu’un prompt de paires (f_i, u_i) est informatif par sa seule longueur : excitation spectrale, diversité et format des tokens sont des variables de premier ordre.

Cette littérature générale fournit le contexte conceptuel, mais la régression linéaire reste le laboratoire où mécanisme, entraînement et risque sont le plus précisément calculables.

3 ICL pour la régression linéaire : fondations empiriques et algorithmiques

3.1 Le paradigme expérimental fondateur

Garg et al. ont établi le banc d’essai moderne : un Transformer est pré-entraîné sur des prompts (x_i, y_i) engendrés par des fonctions nouvelles à chaque tâche, puis évalué sans mise à jour de paramètres sur une fonction non vue [9]. Pour la régression linéaire gaussienne, des Transformers standards atteignent empiriquement une performance proche des moindres carrés ; ils peuvent aussi traiter régression parcimonieuse, petits réseaux et arbres de décision. Ce travail établit une capacité, mais n’identifie pas de manière unique l’algorithme interne.

Akyürek et al. ont ensuite donné trois niveaux de preuve [10] : constructions explicites de gradient descent et de ridge en forme fermée, correspondances empiriques entre prédictions et estimateurs classiques, et décodage d’états internes ressemblant à des poids ou à des moments. La leçon importante est que « le Transformer apprend la régression » n’implique pas nécessairement « chaque couche est un pas de gradient » : le même comportement entrée–sortie peut être réalisé par plusieurs circuits.

3.2 Attention linéaire comme pas de gradient préconditionné

Von Oswald et al. donnent une construction où une couche d’attention linéaire réalise la transformation induite par un pas de gradient sur une perte de régression [11]. Ahn et al. montrent que les paramètres appris peuvent correspondre à un gradient préconditionné dépendant de la distribution des covariables [12]. Mahankali et al. prouvent que, pour une couche d’auto-attention linéaire et des covariables gaussiennes, le minimiseur populationnel implémente précisément un pas de gradient, ou un pas préconditionné en cas d’anisotropie [13].

Ces résultats sont centraux pour Richardson, car le pas

$$z_{\ell+1} = z_\ell + B(c - H z_\ell)$$

est exactement un pas de gradient préconditionné pour une quadratique. Ils ne prouvent cependant pas qu’un Transformer standard entraîné sur des EDP apprend spontanément le bon B , ni que plusieurs couches reproduisent un solveur stable hors distribution.

3.3 Ce que l’entraînement apprend réellement

Zhang, Frei et Bartlett analysent le gradient flow d’une couche linéaire et montrent convergence vers un minimum global et robustesse à certains shifts, mais fragilité aux shifts de covariance des covariables [14]. Les travaux de Wu et al. quantifient le nombre de tâches de pré-entraînement requis pour la régression linéaire [15]. Raventós et al. montrent empiriquement et théoriquement que la diversité des tâches gouverne une transition entre mémorisation de tâches et algorithme non bayésien généralisant à de nouvelles tâches [16]. Li et al. étudient stabilité et généralisation de Transformers vus comme algorithmes [17].

Les résultats de sélection d’algorithme de Bai et al. sont complémentaires : des Transformers suffisamment construits peuvent sélectionner en contexte un algorithme adapté parmi plusieurs classes statistiques [18]. Ils portent surtout sur expressivité et garanties de risque, pas sur l’identification mécanistique d’un modèle effectivement entraîné.

3.4 Au-delà du premier ordre

La conclusion « les Transformers font seulement gradient descent » est trop forte. Fu et al. observent des taux proches d’une itération de Newton et donnent une construction de plusieurs itérations de Newton en profondeur [19]. Vladymyrov et al. montrent la polyvalence de l’attention linéaire comme apprenant en contexte [20]. Pour les fonctions non linéaires, des constructions relient attention et

gradient fonctionnel [21]; pour la régression par noyau gaussien, Yan et al. donnent en 2026 une construction Transformer softmax réalisant une itération de Richardson préconditionnée [33].

Le papier de Yan et al. est particulièrement proche du projet, mais sa portée doit être lue précisément. L’attention softmax produit un opérateur de noyau gaussien normalisé ; des MLP ReLU approximent des opérations scalaires locales (produits, inverse et normalisations). La construction utilise $O(\log(1/\varepsilon))$ blocs et une largeur MLP dépendant de la précision. L’alignement empirique couche–itéré est le plus fort pour Richardson. Cela justifie Richardson comme mécanisme réalisable, mais ne montre pas qu’il est le meilleur solveur à budget de calcul fixé.

4 Profondeur, boucles et conditionnement

4.1 Pourquoi une couche n’est pas une itération universelle

Les constructions d’expressivité fixent généralement des paramètres à la main. Elles prouvent qu’un circuit existe, non qu’il est identifiable par l’entraînement. De plus, les couches peuvent effectuer des sous-opérations différentes : lecture du prompt, agrégation de moments, mise à jour d’état et readout. Compter « une couche = une itération » n’est honnête que si le bloc et l’état ont été définis pour rendre l’égalité exacte.

Le conditionnement spectral rend cette nuance essentielle. Sur une quadratique SPD de condition κ , Richardson optimal demande typiquement $O(\kappa \log(1/\varepsilon))$ itérations, tandis que Chebyshev, HeavyBall optimal et CG présentent une dépendance en $O(\sqrt{\kappa} \log(1/\varepsilon))$ sous leurs hypothèses classiques [67, 68]. Une architecture qui apprend seulement un pas de gradient peut donc être mécaniquement correcte mais numériquement dominée.

4.2 Transformers bouclés et poids partagés

Gatmiry et al. analysent des Transformers linéaires bouclés : le minimum global de la perte populationnelle réalise un gradient préconditionné multi-pas, et le partage des poids impose un biais vers les itérations de point fixe [22]. Leur étude complémentaire montre que profondeur et diversité des tâches interagissent avec le conditionnement, et que les modèles bouclés ont de meilleures propriétés de robustesse et de monotonie en profondeur que certains modèles non partagés [23]. Chen et al. obtiennent également des gains d’efficacité théorique pour des Transformers bouclés réalisant plusieurs pas de gradient [24]. Huang et al. utilisent une chaîne de pensée explicite pour apprendre plusieurs pas de gradient [25].

Pour le projet, le partage des poids est plus qu’une économie de paramètres : il rend plausible une extrapolation en profondeur et permet d’énoncer une loi de récurrence indépendante de L . HeavyBall à coefficients constants s’insère naturellement dans ce cadre. Chebyshev nécessite des coefficients dépendant de l’itération, donc soit des couches non partagées, soit un état de profondeur ou un contrôleur de coefficients.

4.3 Régression structurée, parcimonieuse et mélanges

La régression linéaire isotrope n’épuise pas les structures utiles en EDP. Chen, Zhao et Zou montrent que plusieurs têtes peuvent servir à prétraiter une régression parcimonieuse avant des pas d’optimisation plus simples [26]. Jin et al. analysent les mélanges de régressions, où le Transformer doit implicitement identifier une composante avant d’estimer ses coefficients [27]. Ces modèles sont proches d’une famille d’opérateurs à plusieurs régimes. Les travaux sur les représentations et les sous-espaces faibles dimensions expliquent aussi comment un ICL peut exploiter des caractéristiques partagées plutôt qu’une covariance isotrope [28, 29].

5 Théorie haute dimension : matrices aléatoires et répliques

5.1 Limite proportionnelle de l’attention linéaire

Lu et al. développent une théorie asymptotique exactement soluble pour une couche d’attention linéaire en régression [30]. Dimension des tokens, longueur de contexte et diversité des tâches croissent proportionnellement, tandis que le nombre d’exemples de pré-entraînement suit une autre échelle. La théorie prédit double descente et transition entre mémorisation et véritable ICL. Letey et al. étendent ce point de vue au décalage entre covariance de tâches de pré-entraînement et de test, et introduisent une mesure d’alignement qui prédit la généralisation [31].

Ce régime complète les bornes non asymptotiques : celles-ci contrôlent le pire cas ou une probabilité haute, tandis que la limite proportionnelle vise une valeur précise du risque typique. Les deux niveaux ne sont pas interchangeables.

5.2 Le papier de faible rang 2510.04548

Takanami, Takahashi et Kabashima étudient une couche d’attention linéaire entraînée sur des tâches

$$w^\mu = \sqrt{D/r} A v^\mu, \quad A \in \mathbb{R}^{D \times r},$$

partageant un sous-espace latent de rang r [32]. Leur limite fait croître D , la longueur de contexte, le nombre de tâches de base, le rang et le nombre d’instances en gardant fixes plusieurs rapports :

$$\alpha = L/D, \quad \rho = r/D, \quad \kappa = M_0/D, \quad \gamma = M/(DM_0).$$

Le calcul de répliques prédit une décomposition en loi

$$\hat{y} \stackrel{d}{=} \hat{y}_{\text{algo}} + \hat{y}_{\text{mem}} + \hat{y}_{\text{struct}}.$$

Le terme algorithmique projette un estimateur de type matched filter sur le sous-espace appris $AA^\top$; les deux autres termes représentent un bruit lié aux tâches mémorisées et un bruit lié au défaut d’alignement avec la structure. Le papier prédit aussi :

- une transition lorsque la diversité de tâches devient suffisante par rapport au rang latent ;
- un compromis spécialisation–robustesse OOD ;
- une régularisation implicite d’ordre $1/\alpha$ créée par les fluctuations d’un contexte de pré-entraînement fini ;
- un accord numérique des paramètres d’ordre et du risque avec les expériences, plus une vérification qualitative sur attention softmax.

Il faut toutefois employer la terminologie des auteurs : il s’agit de *Results* issus de la méthode des répliques, et non de théorèmes rigoureux, car l’argument utilise continuation analytique, échange de limites et ansatz de symétrie des répliques. Le résultat est extrêmement pertinent pour modéliser la phase d’encodeur ou l’apprentissage d’un sous-espace de préconditionnement. Il ne traite pas :

1. une attention profonde ou bouclée ;
2. une matrice de Galerkin ou un opérateur SPD dépendant de la tâche ;
3. une propagation d’erreur de $\hat{A}$ vers $\hat{A}^{-1}f$;
4. un polynôme de solveur Richardson, HeavyBall, Chebyshev ou CG ;
5. le coût ou la stabilité d’un préconditionneur appris.

La bonne utilisation dans le projet est donc de reprendre sa machinerie pour la loi de l’encodeur ou du préconditionneur, puis de composer cette loi avec un calcul spectral exact du décodeur. Le risque du décodeur fixe peut souvent s’écrire comme une intégrale spectrale pondérée ; c’est ce chaînon qui manque au papier de faible rang.

5.3 Rigueur possible au-delà des répliques

La méthode des répliques est un calcul heuristique puissant, mais certaines de ses prédictions peuvent être rendues rigoureuses par interpolation adaptative dans des modèles bayésiens ou par état d'évolution AMP [69, 70]. Pour un projet de solveur, trois niveaux de résultat peuvent coexister proprement :

1. un théorème déterministe conditionnel en fonction du spectre de $B_\theta \hat{A}$;
2. une borne probabiliste non asymptotique pour l'encodeur ;
3. une prédiction replica/RMT précise du risque typique en limite proportionnelle, explicitement annoncée comme telle.

Cette séparation évite de transformer une formule de répliques en garantie uniforme injustifiée.

6 Des systèmes linéaires aux opérateurs et aux EDP

6.1 ICL de systèmes linéaires

Cole, Lu, Xu et Zhang étudient directement l'ICL de systèmes linéaires avec une couche de Transformer linéaire [34]. Ils donnent des lois d'échelle en fonction du nombre de tâches de pré-entraînement N , de la longueur d'entraînement n et de la longueur de prompt test m , ainsi qu'une condition de diversité nécessaire et suffisante dans leur cadre pour certains shifts de tâches. Leur théorème abstrait transfère le résultat fini-dimensionnel à l'apprentissage d'opérateurs et donne une application aux EDP elliptiques linéaires après projection de Galerkin.

Ce travail est la référence théorique la plus proche, mais le modèle est un prédicteur linéaire à une couche, pas un solveur récurrent explicitement analysé. Sa borne regroupe l'effet statistique et l'approximation de discrétisation ; elle ne compare pas les complexités en profondeur de Richardson, HeavyBall, Chebyshev et PCG, et n'apprend pas explicitement un préconditionneur SPD conditionné par le prompt.

Le prolongement aux systèmes dynamiques montre par ailleurs une séparation de profondeur : des Transformers multi-couches atteignent l'échelle d'un estimateur moindres carrés alors qu'une classe à une couche conserve une erreur non nulle sur données non-i.i.d. [35]. Des travaux voisins interprètent les Transformers comme estimateurs d'état implicites ou étudient la représentation du filtre de Kalman [36, 37]. Ils confirment qu'un contexte corrélé change qualitativement le problème par rapport à la régression i.i.d.

6.2 Operator learning classique

DeepONet sépare un réseau *branch*, qui encode la fonction d'entrée échantillonnée, et un réseau *trunk*, qui encode le point de sortie [38]. FNO paramètre l'opérateur intégral dans l'espace de Fourier et vise une certaine invariance à la résolution [39]. Le cadre neural operator général formalise des couches entre espaces de fonctions et leurs propriétés d'approximation [40]. Des estimations d'erreur pour DeepONet séparent encodage, approximation et reconstruction [41] ; des bornes de complexité de Rademacher ont aussi été développées pour FNO [42].

Ces travaux apprennent habituellement une loi amortie sur une distribution d'opérateurs ou de paramètres. Ils ne réalisent pas nécessairement une adaptation ICL à un opérateur entièrement nouveau à partir de quelques paires (f_i, u_i) .

6.3 Transformers comme neural operators

Le Galerkin Transformer montre que l'attention linéarisée sans softmax peut être interprétée comme une projection de type Petrov–Galerkin et l'applique à des problèmes directs et à une identification de coefficient d'interface [43]. OFormer développe une architecture Transformer pour les opérateurs spatio-temporels [44]. GNOT traite entrées hétérogènes, maillages irréguliers et décomposition

géométrie souple [45]. Transolver construit des tokens physiques et une attention efficace pour géométries générales [46].

Ces architectures sont des surrogates puissants ; leur attention n’implique pas à elle seule une itération de Galerkin ou de Krylov. La ressemblance entre attention et projection variationnelle est structurelle, pas une preuve de convergence vers la solution d’un système donné.

6.4 Pré-entraînement multi-EDP et modèles de fondation

DPOT combine Transformer d’opérateur, attention de Fourier et pré-entraînement auto-régressif débruitant sur de nombreuses trajectoires [47]. Poseidon utilise un Transformer d’opérateur multi-échelle et la propriété de semi-groupe pour pré-entraînement et transfert sur plusieurs EDP [48]. PDEformer représente symboliquement l’EDP comme un graphe, traité par Transformer, puis décode une représentation implicite de la solution ; il fournit aussi des expériences d’identification de coefficients [49]. UPS et Unisolver visent également l’adaptation multi-physique et les changements de modalité [50, 51].

Leurs résultats portent surtout sur accuracy, sample efficiency, zero-shot ou fine-tuning. Ils ne donnent généralement pas une décomposition complète

discrétisation + identification de loi + solveur fini + méta-généralisation.

Ils constituent en revanche des baselines empiriques nécessaires si la contribution revendique un avantage pratique général en EDP.

6.5 In-context operator learning

ICON reçoit des démonstrations fonctionnelles comme prompt et peut réaliser différentes prédictions directes ou inverses sans fine-tuning [52]. Une étude sur les lois de conservation 1D montre une généralisation à certaines formes d’EDP non vues, mais aussi la dépendance au design du prompt et au domaine de tâches [53]. Cole et al. fournissent la première justification mathématique directe de ce paradigme pour des opérateurs linéaires [34]. Mishra, Patel et Tewari montrent enfin que des Continuum Transformers peuvent réaliser un gradient d’opérateur en contexte [54].

Cette branche est la plus proche du projet au niveau de la sémantique du prompt. La lacune demeure le décodeur numérique : ICON apprend une règle de prédiction, alors que le projet cherche à exposer et accélérer la résolution du système conditionné par le prompt.

7 Problèmes inverses en EDP et architectures Transformer

7.1 Surrogates inverses et opérateurs inverses

Les problèmes inverses en EDP sont mal posés : stabilité conditionnelle, régularisation et bruit d’observation ne peuvent être ignorés [65, 66]. Un réseau qui prédit un coefficient à partir d’une solution n’est donc pas seulement un solveur de système linéaire ; il apprend un inverse régularisé dépendant du prior et du modèle de bruit.

ViTO utilise une architecture Vision Transformer–Operator pour des problèmes inverses paramétriques [55]. Les Neural Inverse Operators apprennent directement une application données–paramètre pour des inversions d’EDP [56]. BiLO imbrique un opérateur local dans une optimisation bi-niveau pour l’identification de paramètres [57]. LNO réalise des calculs directs et inverses dans un espace latent via cross-attention physique [58]. PDEformer inclut aussi la récupération de coefficients [49].

Ces méthodes doivent être comparées sur l’identification de a ou V , mais elles ne répondent pas automatiquement à la question du meilleur décodeur de $\hat{A}u = f$. Inverse statistique et solveur numérique doivent rester deux étages auditables.

7.2 Formulation faible et identification de faible dimension

Si

$$a_z(x) = a_0(x) + \sum_{k=1}^K z_k \psi_k(x),$$

les paires de prompt (f_i, u_i) et des fonctions test v_r donnent

$$\sum_{k=1}^K z_k \int_{\Omega} \psi_k \nabla u_i \cdot \nabla v_r = \int_{\Omega} f_i v_r - \int_{\Omega} a_0 \nabla u_i \cdot \nabla v_r.$$

Après empilement, $G_m z \simeq b_m$. Cette réduction relie le problème inverse elliptique à une régression linéaire structurée, mais les lignes de G_m ne sont ni isotropes ni indépendantes de la tâche. Leur covariance dépend de la PDE, du choix de tests, de l’excitation des modes par les sources et du bruit sur u_i . Les théories ICL gaussiennes ne s’appliquent donc pas par simple substitution ; il faut contrôler ou modéliser ce design structuré.

8 Préconditionnement appris et solveurs hybrides

8.1 Pourquoi cette littérature est une baseline obligatoire

La littérature « learned solver » adopte souvent une stratégie plus sûre que l’imitation complète d’un algorithme : le réseau produit un warm start, une correction de grille grossière, une factorisation approchée ou un préconditionneur ; un solveur classique impose ensuite la précision finale. Cette séparation offre des résidus vérifiables et permet de conserver des garanties numériques.

Sappl et al. apprennent des préconditionneurs pour CG [59]. Li et al. apprennent un préconditionneur de CG pour des systèmes issus d’EDP [60]. NeuralIF produit par GNN une factorisation incomplète parcimonieuse et l’entraîne via un proxy Frobenius efficace [61]. FCG-NO utilise un neural operator potentiellement non linéaire comme préconditionneur d’un flexible CG, avec transfert entre résolutions [62]. UGrid apprend un schéma multigrille avec contrôle de convergence [63]. Les travaux de multigrille appris remontent au moins à l’apprentissage de prolongations et de coarse operators par Greenfeld et al. [64].

Ces méthodes sont les vraies baselines d’une revendication « meilleur solveur decoder ». FNO, DeepONet ou ICON seuls ne suffisent pas comme comparateurs : il faut aussi mesurer contre Jacobi/ILU/AMG, CG/PCG, Chebyshev, warm start neural et préconditionneur neural + Krylov.

8.2 Trois architectures possibles pour le projet

A. Transformer pur à polynôme fixe. Le bloc partagé conserve $(z_\ell, z_{\ell-1})$ et applique

$$z_{\ell+1} = z_\ell + \alpha B_\theta(\mathcal{C})(c - H z_\ell) + \beta(z_\ell - z_{\ell-1}). \quad (2)$$

L’attention calcule le produit prompt–vecteur ou l’agrégation faible ; le mélange linéaire des canaux réalise les coefficients scalaires. Aucune division adaptative n’est requise. Avec $\beta = 0$, on retrouve Richardson. Avec $\beta > 0$, on obtient HeavyBall. Des coefficients dépendant de ℓ donnent une semi-itération de Chebyshev.

B. Transformer-conditioned PCG. Le Transformer produit un préconditionneur SPD, par exemple

$$B_\theta(\mathcal{C}) = D^{-1/2}(I + U_\theta(\mathcal{C})U_\theta(\mathcal{C})^\top)D^{-1/2},$$

puis une cellule PCG externe effectue exactement les produits scalaires, quotients et conjuguaisons. C’est probablement la meilleure architecture pratique, mais elle doit être nommée *Transformer-conditioned PCG* et non « vanilla Transformer implements PCG ».

C. Transformer pur imitant PCG. Il faut stocker résidu, résidu préconditionné, direction, plusieurs produits scalaires et des quotients dépendant du prompt et de l'itération. Une construction par approximation MLP est possible sur un domaine dont les dénominateurs sont séparés de zéro, mais elle ajoute une erreur arithmétique et plusieurs sous-blocs par itération. Cette voie est théoriquement originale, mais moins naturelle pour le premier démonstrateur.

8.3 HeavyBall, Chebyshev et PCG : comparaison honnête

Pour $S = B^{1/2}HB^{1/2}$ avec spectre dans $[\mu, L_s]$, HeavyBall constant obéit mode par mode à

$$e_{\ell+1}(\lambda) = (1 + \beta - \alpha\lambda)e_\ell(\lambda) - \beta e_{\ell-1}(\lambda).$$

Les paramètres quadratiques classiques sont

$$\alpha_\star = \frac{4}{(\sqrt{L_s} + \sqrt{\mu})^2}, \quad \beta_\star = \left(\frac{\sqrt{L_s} - \sqrt{\mu}}{\sqrt{L_s} + \sqrt{\mu}} \right)^2.$$

Le taux asymptotique modal est $q = (\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1)$, mais une borne finie doit conserver le facteur transitoire lié aux racines doubles aux extrémités. Chebyshev minimise un polynôme sur un intervalle spectral connu et constitue la meilleure baseline polynomiale non adaptative. PCG choisit ses coefficients en fonction du second membre et du sous-espace de Krylov ; il reste la baseline supérieure pour SPD, mais sa non-linéarité en f complique une théorie replica du décodeur.

Le meilleur compromis scientifique est donc : HeavyBall partagé comme contribution « Transformer pur », Chebyshev comme contrôle polynomial, PCG comme oracle numérique, et Transformer-conditioned PCG comme architecture hybride orientée performance.

9 Décomposition des erreurs : ce qu'une théorie complète doit prouver

Soit A l'opérateur discret vrai, $\hat{A}$ l'opérateur encodé, et $\hat{u}_L$ la sortie après L boucles. Une décomposition minimale est

$$\begin{aligned} \mathbb{E}\|\hat{u}_L - u\|_{\mathcal{H}}^2 \lesssim & \underbrace{\varepsilon_{\text{disc}}(d)}_{\text{Galerkin/maillage}} + \underbrace{\varepsilon_{\text{inv}}(m, \sigma, K)}_{\text{identification de loi}} \\ & + \underbrace{\varepsilon_{\text{meta}}(N, n)}_{\text{pré-entraînement}} + \underbrace{\varepsilon_{\text{solver}}(L, B_\theta \hat{A})}_{\text{profondeur finie}} + \underbrace{\varepsilon_{\text{arch}}}_{\text{réalisation par attention}}. \end{aligned} \quad (3)$$

Pour un décodeur polynomial fixe, conditionnellement à $\hat{A}$ et B_θ , la sortie est linéaire en f . Si p_L est son polynôme résiduel,

$$\hat{u}_L = \widehat{A}^{-1}[I - p_L(B_\theta \hat{A})]f.$$

Pour $f \sim \mathcal{N}(0, \Sigma_f)$, le risque conditionnel du solveur admet une trace exacte

$$\mathbb{E}_f \|\hat{u}_L - \hat{A}^{-1}f\|_2^2 = \text{tr}(\Sigma_f E_L^\top E_L), \quad E_L = \hat{A}^{-1}p_L(B_\theta \hat{A}).$$

Cette formule rend HeavyBall/Chebyshev compatibles avec une intégration RMT ou replica de la loi spectrale de l'encodeur. PCG, dont les coefficients dépendent de f , demande des paramètres d'ordre supplémentaires sur le processus de Krylov.

La propagation opérateur-solution suit l'identité de résolvante

$$\hat{A}^{-1} - A^{-1} = \hat{A}^{-1}(A - \hat{A})A^{-1}.$$

Sous coercivité uniforme $A, \hat{A} \succeq a_0 I$, une borne d'opérateur pour $\hat{A} - A$ devient une borne de solution avec un facteur a_0^{-2} . Ce pont est indispensable : une bonne perte Frobenius sur l'opérateur n'est pas automatiquement une bonne erreur de solution si les erreurs sont alignées avec des modes spectraux fragiles.

10 Tableau critique de la littérature la plus proche

Référence	Objet	Résultat central	Ce qui manque pour notre question
Garg et al. [9]	ICL régression	Transformer entraîné proche de moindres carrés	mécanisme unique, EDP, solveur spectral
Akyürek et al. [10]	algorithmes ICL	constructions GD, ridge, OLS	apprentissage d'un opérateur PDE et stabilité OOD
Ahn et al. [12]	régression anisotrope	gradient préconditionné	profondeur accélérée et contrainte SPD
Fu et al. [19]	second ordre	construction et alignement Newton	arithmétique, EDP et erreur encodeur-décodeur
Gatmiry et al. [22]	Transformer bouclé	multi-pas GD appris	accélération HB/Chebyshev et problème inverse
Lu et al. [30]	théorie haute dimension	risque exact, double descente, diversité	opérateur SPD, profondeur et solveur
Takanami et al. [32]	tâches de faible rang	décomposition replica et transition de phase	décodeur profond et inversion A^{-1}
Cole et al. [34]	systèmes linéaires/EDP	bornes ID/OOD et transfert opérateur	solveur récurrent accéléré et préconditionneur appris
Yan et al. [33]	KRR softmax	Richardson constructif avec garantie	meilleur solveur, EDP et apprentissage SPD
ICON [52]	ICL d'opérateur	adaptation sans fine-tuning	garantie de solveur et coût spectral
Poseidon/DPOT [48, 47]	fondation EDP	transfert empirique multi-physique	mécanisme de résolution et théorie de risque
NeuralIF [61]	préconditionneur CG	factorisation apprise efficace	ICL par prompt et théorie meta/OOD
FCG-NO [62]	neural operator + FCG	préconditionneur non linéaire multi-résolution	Transformer pur et décomposition replica

11 La lacune précise et la question que nous améliorons

11.1 Lacune

Aucune des branches examinées ne réunit simultanément les cinq propriétés suivantes :

1. une tâche est un opérateur elliptique caché, identifié uniquement par un prompt de paires source-solution ;
2. le modèle se généralise à un nouvel opérateur sans mise à jour de ses poids ;
3. le décodeur est un solveur accéléré explicite, bouclé et à poids partagés, plutôt qu'un surrogate opaque ;
4. le préconditionnement est appris à partir du prompt avec une propriété SPD ou une stabilité spectrale vérifiable ;

5. la théorie sépare erreur de Galerkin, erreur d’encodeur, erreur de profondeur, réalisation architecturale et généralisation ID/OOD, avec en plus une prédiction haute dimension du risque typique.

11.2 Question principale proposée

Peut-on pré-entraîner un Transformer bouclé qui, à partir d’un prompt de paires source–solution générées par un opérateur elliptique inconnu, identifie les directions spectrales utiles et applique à un nouveau second membre un décodeur préconditionné accéléré à poids partagés, de sorte que (i) sa trajectoire interne réalise exactement ou uniformément Heavy-Ball/Chebyshev, (ii) son erreur totale admet une décomposition non asymptotique encodeur–solveur, (iii) son risque typique en grande dimension est prédit par RMT/répliques, et (iv) il surpasse Richardson et les neural operators directs à budget égal tout en approchant PCG ?

Cette formulation améliore la question de Cole et al. en remplaçant la seule prédiction d’un système linéaire par une architecture encodeur–solveur explicite et accélérée. Elle améliore la ligne von Oswald–Ahn–Gatmiry en passant d’une régression synthétique à un design faible issu d’EDP et en distinguant apprentissage du préconditionneur et itération. Elle améliore la ligne Takanami–Lu en ajoutant une dynamique de décodeur profonde dont le polynôme spectral est observable. Enfin, elle se distingue de FCG-NO en imposant adaptation ICL à une loi cachée et en visant un circuit Transformer pur avant l’hybride PCG.

11.3 Sous-questions falsifiables

1. **Réalisation.** Un bloc partagé sans MLP arithmétique peut-il réaliser l’agrégation $G_m^\top(b_m - G_m z_\ell)$ et la mémoire HeavyBall avec une erreur uniforme contrôlée ?
2. **Apprentissage spectral.** Le préconditionneur appris réduit-il réellement $\text{cond}(B_\theta^{1/2} H B_\theta^{1/2})$, ou seulement la perte sur la distribution de seconds membres vue ?
3. **Structure de faible rang.** Le modèle retrouve-t-il un sous-espace de correction analogue au $AA^\top$ de Takanami et al., et existe-t-il une transition en fonction du rang latent, de la diversité et du nombre de prompts ?
4. **Robustesse.** Les coefficients partagés restent-ils stables sous shift de maillage, covariance des sources, coefficients d’EDP et condition number ?
5. **Performance.** À nombre égal de produits Hv , HeavyBall appris bat-il Richardson ; Chebyshev oracle borne-t-il le meilleur polynôme fixe ; combien reste-t-il d’écart à PCG, AMG et neural-preconditioned CG ?
6. **Théorie.** La loi replica de l’encodeur, composée avec le polynôme du décodeur, prédit-elle le risque couche par couche et ses pics transitoires ?

12 Contributions réalistes et ordre de travail

Une hiérarchie crédible de contributions est la suivante.

Contribution 1 : mécanisme exact. Construire un état de tokens contenant $(z_\ell, z_{\ell-1})$, des tokens de prompt faibles et éventuellement des tokens spectraux. Prouver qu’un bloc partagé implémente un pas de HeavyBall préconditionné, modulo l’erreur déjà nécessaire pour le produit prompt–vecteur. Cette construction ne requiert ni inverse scalaire adaptatif ni approximation de quotient.

Contribution 2 : transfert encodeur–décodeur. Établir un théorème conditionnel qui compose : coercivité et Galerkin, concentration de $\hat{A}$, stabilité de B_θ , polynôme résiduel p_L , puis risque de solution. Comparer la borne et la trace exacte à Richardson.

Contribution 3 : théorie haute dimension. Adapter le modèle de faible rang aux lignes de design faible G_m , obtenir la loi du sous-espace ou du préconditionneur appris, puis intégrer $p_L(\lambda)^2$ contre la mesure spectrale limite. Cette partie peut être annoncée comme prédiction replica, avec une validation numérique systématique.

Contribution 4 : benchmark solveur. Comparer, à même nombre de produits matrice–vecteur et même budget mémoire : Richardson scalaire, Jacobi–Richardson, HeavyBall appris/oracle, Chebyshev, CG/PCG, AMG/ILU, NeuralIF ou préconditionneur neuronal, FCG-NO, ainsi que des surrogates directs FNO/GNOT/Poseidon lorsque le jeu de données le permet.

Contribution 5 : hybride performance. Une fois le mécanisme pur établi, entraîner un Transformer qui produit un préconditionneur SPD pour un PCG externe. Cette version peut viser la meilleure performance pratique sans diluer la contribution théorique du décodeur pur.

13 Protocole expérimental minimal pour soutenir les revendications

13.1 Axes de données

Les expériences doivent varier séparément : dimension de Galerkin d , rang latent K , longueur du prompt m , nombre de tâches N , bruit, condition number, covariance des sources, diversité des opérateurs, géométrie et résolution. Les shifts ID/OOD doivent inclure :

- nouvelles réalisations du même prior de coefficients ;
- rang ou longueur de corrélation hors entraînement ;
- contraste elliptique plus élevé ;
- maillage ou base différents ;
- nouvelles covariances de seconds membres ;
- combinaisons de directions spectrales rares.

13.2 Métriques

La seule MSE de sortie est insuffisante. Il faut rapporter :

1. erreur coefficient/opérateur en Frobenius et norme opérateur ;
2. erreur solution L^2 et énergie H^1 ;
3. résidu relatif et erreur par produit Hv ;
4. spectre de $B_\theta^{1/2}HB_\theta^{1/2}$ et conditionnement effectif ;
5. stabilité couche par couche, facteur transitoire et extrapolation en nombre de boucles ;
6. coût total, mémoire, paramètres, temps de préconditionnement et temps jusqu’à une tolérance ;
7. calibration d’incertitude si l’inverse statistique est revendiqué.

13.3 Tests mécanistiques

La trajectoire cachée doit être projetée vers les itérés Richardson, HeavyBall, Chebyshev, CG et Newton sur exactement les mêmes prompts. Une corrélation linéaire seule ne suffit pas : comparer aussi les résidus, les polynômes spectraux par mode, les mises à jour et les contrefactuels obtenus en modifiant le spectre sans changer les vecteurs propres. Des ablations doivent retirer la mémoire $z_{\ell-1}$, partager ou non les poids, figer ou apprendre B_θ , et changer la distribution des seconds membres.

14 Conclusion

La littérature soutient trois constats robustes. Premièrement, les Transformers peuvent apprendre ou représenter des algorithmes de régression en contexte, du gradient préconditionné à des méthodes plus rapides. Deuxièmement, les Transformers et neural operators sont devenus des surrogates puissants pour les EDP et peuvent parfois s’adapter sans fine-tuning. Troisièmement, les approches hybrides réseau–Krylov sont aujourd’hui les concurrentes naturelles lorsque la précision numérique finale compte.

La contribution non couverte n’est donc pas « utiliser un Transformer pour une EDP » ni « montrer qu’il peut faire Richardson ». Elle est de construire et analyser un *solveur-décodeur en contexte accéléré* : l’opérateur caché et son préconditionnement sont appris à partir du prompt, la dynamique de résolution est explicite et bouclée, les erreurs statistiques et numériques sont séparées, et la théorie haute dimension prédit comment la structure de faible rang apprise se transforme en convergence du solveur. HeavyBall fournit le premier mécanisme pur le plus simple ; Chebyshev est le contrôle polynomial nécessaire ; PCG reste la baseline numérique et le débouché hybride.

Références

- [1] T. B. Brown, B. Mann, N. Ryder, et al. Language Models Are Few-Shot Learners. *NeurIPS*, 2020. [arXiv:2005.14165](#).
- [2] S. M. Xie, A. Raghunathan, P. Liang, and T. Ma. An Explanation of In-Context Learning as Implicit Bayesian Inference. *ICLR*, 2022. [arXiv:2111.02080](#).
- [3] D. Dai, Y. Sun, L. Dong, et al. Why Can GPT Learn In-Context? Language Models Implicitly Perform Gradient Descent as Meta-Optimizers. 2023. [arXiv:2212.10559](#).
- [4] C. Olsson, N. Elhage, N. Nanda, et al. In-Context Learning and Induction Heads. 2022. [arXiv:2209.11895](#).
- [5] S. C. Y. Chan, A. Santoro, A. K. Lampinen, et al. Data Distributional Properties Drive Emergent In-Context Learning in Transformers. *NeurIPS*, 2022. [arXiv:2205.05055](#).
- [6] S. C. Y. Chan, I. Dasgupta, J. Kim, et al. Transformers Generalize Differently from Information Stored in Context vs in Weights. 2022. [arXiv:2210.05675](#).
- [7] S. Min, X. Lyu, A. Holtzman, et al. Rethinking the Role of Demonstrations: What Makes In-Context Learning Work? *EMNLP*, 2022. [arXiv:2202.12837](#).
- [8] N. Wies, Y. Levine, and A. Shashua. The Learnability of In-Context Learning. *NeurIPS*, 2023. [arXiv:2303.07895](#).
- [9] S. Garg, D. Tsipras, P. Liang, and G. Valiant. What Can Transformers Learn In-Context? A Case Study of Simple Function Classes. *NeurIPS*, 2022. [arXiv:2208.01066](#).
- [10] E. Akyürek, D. Schuurmans, J. Andreas, T. Ma, and D. Zhou. What Learning Algorithm Is In-Context Learning? Investigations with Linear Models. *ICLR*, 2023. [arXiv:2211.15661](#).
- [11] J. von Oswald, E. Niklasson, E. Randazzo, et al. Transformers Learn In-Context by Gradient Descent. *ICML*, 2023. [arXiv:2212.07677](#).
- [12] K. Ahn, X. Cheng, H. Daneshmand, and S. Sra. Transformers Learn to Implement Preconditioned Gradient Descent for In-Context Learning. *NeurIPS*, 2023. [arXiv:2306.00297](#).
- [13] A. Mahankali, T. B. Hashimoto, and T. Ma. One Step of Gradient Descent Is Provably the Optimal In-Context Learner with One Layer of Linear Self-Attention. 2023. [arXiv:2307.03576](#).
- [14] R. Zhang, S. Frei, and P. L. Bartlett. Trained Transformers Learn Linear Models In-Context. 2023. [arXiv:2306.09927](#).
- [15] J. Wu, D. Zou, Z. Chen, V. Braverman, Q. Gu, and P. L. Bartlett. How Many Pretraining Tasks Are Needed for In-Context Learning of Linear Regression? 2024. [arXiv:2310.08391](#).
- [16] A. Raventós, M. Paul, F. Chen, and S. Ganguli. Pretraining Task Diversity and the Emergence of Non-Bayesian In-Context Learning for Regression. *NeurIPS*, 2023. [arXiv:2306.15063](#).
- [17] Y. Li, M. E. Ildiz, D. Papailiopoulos, and S. Oymak. Transformers as Algorithms: Generalization and Stability in In-Context Learning. *ICML*, 2023. [arXiv:2301.07067](#).
- [18] Y. Bai, F. Chen, H. Wang, C. Xiong, and S. Mei. Transformers as Statisticians: Provable In-Context Learning with In-Context Algorithm Selection. *NeurIPS*, 2023. [arXiv:2306.04637](#).
- [19] D. Fu, T.-Q. Chen, R. Jia, and V. Sharan. Transformers Learn to Achieve Second-Order Convergence Rates for In-Context Linear Regression. 2024. [arXiv:2310.17086](#).
- [20] M. Vladymyrov, J. von Oswald, M. Sandler, and R. Ge. Linear Transformers Are Versatile In-Context Learners. 2024. [arXiv:2402.14180](#).
- [21] X. Cheng, Y. Chen, and S. Sra. Transformers Implement Functional Gradient Descent to Learn Non-Linear Functions In Context. 2024. [arXiv:2312.06528](#).

- [22] K. Gatmiry, N. Saunshi, S. J. Reddi, S. Jegelka, and S. Kumar. Can Looped Transformers Learn to Implement Multi-step Gradient Descent for In-context Learning? 2024. [arXiv:2410.08292](#).
- [23] K. Gatmiry, N. Saunshi, S. J. Reddi, S. Jegelka, and S. Kumar. On the Role of Depth and Looping for In-Context Learning with Task Diversity. 2024. [arXiv:2410.21698](#).
- [24] B. Chen, X. Li, Y. Liang, and Z. Shi. Bypassing the Exponential Dependency: Looped Transformers Efficiently Learn In-Context by Multi-step Gradient Descent. 2024. [arXiv:2410.11268](#).
- [25] J. Huang, Z. Wang, and J. D. Lee. Transformers Learn to Implement Multi-step Gradient Descent with Chain of Thought. 2025. [arXiv:2502.21212](#).
- [26] X. Chen, L. Zhao, and D. Zou. How Transformers Utilize Multi-Head Attention in In-Context Learning? A Case Study on Sparse Linear Regression. 2024. [arXiv:2408.04532](#).
- [27] Y. Jin, K. Balasubramanian, and L. Lai. In-context Learning for Mixture of Linear Regressions: Existence, Generalization and Training Dynamics. 2024. [arXiv:2410.14183](#).
- [28] T. Guo, W. Hu, S. Mei, et al. How Do Transformers Learn In-Context Beyond Simple Functions? A Case Study on Learning with Representations. 2023. [arXiv:2310.10616](#).
- [29] S. M. Kwon, A. S. Xu, C. Yaras, et al. Out-of-Distribution Generalization of In-Context Learning: A Low-Dimensional Subspace Perspective. 2025. [arXiv:2505.14808](#).
- [30] Y. M. Lu, M. I. Letey, J. A. Zavatone-Veth, A. Maiti, and C. Pehlevan. Asymptotic Theory of In-Context Learning by Linear Attention. *PNAS*, 2025. [arXiv:2405.11751](#).
- [31] M. I. Letey, J. A. Zavatone-Veth, Y. M. Lu, and C. Pehlevan. Pretrain-Test Task Alignment Governs Generalization in In-Context Learning. 2025/2026. [arXiv:2509.26551](#).
- [32] K. Takanami, T. Takahashi, and Y. Kabashima. Learning Linear Regression with Low-Rank Tasks In-Context. *AISTATS*, 2026. [arXiv:2510.04548](#).
- [33] M. Yan, D. Li, C. Kulick, and S. Tang. Transformers Can Implement Preconditioned Richardson Iteration for In-Context Gaussian Kernel Regression. 2026. [arXiv:2605.08475](#).
- [34] F. Cole, Y. Lu, W. Xu, and T. Zhang. In-Context Learning of Linear Systems: Generalization Theory and Applications to Operator Learning. 2024/2025. [arXiv:2409.12293](#).
- [35] F. Cole, Y. Zhao, Y. Lu, and T. Zhang. In-Context Learning of Linear Dynamical Systems with Transformers: Approximation Bounds and Depth-Separation. 2025. [arXiv:2502.08136](#).
- [36] G. Goel and P. Bartlett. Can a Transformer Represent a Kalman Filter? *L4DC*, 2024. [PMLR 242](#).
- [37] U. Akram and H. Vikalo. Transformers as Implicit State Estimators: In-Context Learning in Dynamical Systems. 2024. [arXiv:2410.16546](#).
- [38] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis. Learning Nonlinear Operators via DeepONet Based on the Universal Approximation Theorem of Operators. *Nature Machine Intelligence*, 2021. [arXiv:1910.03193](#).
- [39] Z. Li, N. Kovachki, K. Azizzadenesheli, et al. Fourier Neural Operator for Parametric Partial Differential Equations. *ICLR*, 2021. [arXiv:2010.08895](#).
- [40] N. Kovachki, Z. Li, B. Liu, et al. Neural Operator: Learning Maps Between Function Spaces with Applications to PDEs. *JMLR*, 2023. [arXiv:2108.08481](#).
- [41] S. Lanthaler, S. Mishra, and G. E. Karniadakis. Error Estimates for DeepONets: A Deep Learning Framework in Infinite Dimensions. *Transactions of Mathematics and Its Applications*, 2022. [arXiv:2102.09618](#).
- [42] T. Kim and M. Kang. Bounding the Rademacher Complexity of Fourier Neural Operators. 2022. [arXiv:2209.05150](#).
- [43] S. Cao. Choose a Transformer: Fourier or Galerkin. *NeurIPS*, 2021. [arXiv:2105.14995](#).

- [44] Z. Li, K. Meidani, and A. B. Farimani. Transformer for Partial Differential Equations’ Operator Learning. 2022. [arXiv:2205.13671](#).
- [45] Z. Hao, Z. Wang, H. Su, et al. GNOT: A General Neural Operator Transformer for Operator Learning. *ICML*, 2023. [arXiv:2302.14376](#).
- [46] H. Wu, H. Luo, H. Wang, et al. Transolver: A Fast Transformer Solver for PDEs on General Geometries. *ICML*, 2024. [arXiv:2402.02366](#).
- [47] Z. Hao, C. Su, S. Liu, et al. DPOT: Auto-Regressive Denoising Operator Transformer for Large-Scale PDE Pre-Training. 2024. [arXiv:2403.03542](#).
- [48] M. Herde, B. Raonić, T. Rohner, et al. Poseidon: Efficient Foundation Models for PDEs. 2024. [arXiv:2405.19101](#).
- [49] Z. Ye, X. Huang, L. Chen, et al. PDEformer: Towards a Foundation Model for One-Dimensional Partial Differential Equations. 2024. [arXiv:2402.12652](#).
- [50] J. Shen, T. Marwah, and A. Talwalkar. UPS: Efficiently Building Foundation Models for PDE Solving via Cross-Modal Adaptation. 2024. [arXiv:2403.07187](#).
- [51] H. Zhou, Y. Ma, H. Wu, H. Wang, and M. Long. Unisolver: PDE-Conditional Transformers Towards Universal Neural PDE Solvers. 2024. [arXiv:2405.17527](#).
- [52] L. Yang, S. Liu, T. Meng, and S. J. Osher. In-Context Operator Learning with Data Prompts for Differential Equation Problems. *PNAS*, 2023/2024. [arXiv:2304.07993](#).
- [53] L. Yang and S. J. Osher. PDE Generalization of In-Context Operator Networks: A Study on 1D Scalar Nonlinear Conservation Laws. 2024. [arXiv:2401.07364](#).
- [54] A. Mishra, Y. Patel, and A. Tewari. Continuum Transformers Perform In-Context Learning by Operator Gradient Descent. 2025. [arXiv:2505.17838](#).
- [55] O. Ovadia, A. Kahana, P. Stinis, E. Turkel, and G. E. Karniadakis. ViTO: Vision Transformer-Operator. 2023. [arXiv:2303.08891](#).
- [56] R. Molinaro, Y. Yang, B. Engquist, and S. Mishra. Neural Inverse Operators for Solving PDE Inverse Problems. 2023. [arXiv:2301.11167](#).
- [57] R. Z. Zhang, C. E. Miles, X. Xie, and J. S. Lowengrub. BiLO: Bilevel Local Operator Learning for PDE Inverse Problems. 2024. [arXiv:2404.17789](#).
- [58] T. Wang and C. Wang. Latent Neural Operator for Solving Forward and Inverse PDE Problems. 2024. [arXiv:2406.03923](#).
- [59] J. Sappl, L. Seiler, M. Harders, and W. Rauch. Deep Learning of Preconditioners for Conjugate Gradient Solvers in Urban Water Related Problems. 2019. [arXiv:1906.06925](#).
- [60] Y. Li, P. Y. Chen, T. Du, and W. Matusik. Learning Preconditioner for Conjugate Gradient PDE Solvers. 2023. [arXiv:2305.16432](#).
- [61] P. Häusner, O. Öktem, and J. Sjölund. Neural Incomplete Factorization: Learning Preconditioners for the Conjugate Gradient Method. 2023. [arXiv:2305.16368](#).
- [62] A. Rudikov, V. Fanaskov, E. Muravleva, Y. Laevsky, and I. Oseledets. Neural Operators Meet Conjugate Gradients: The FCG-NO Method for Efficient PDE Solving. 2024. [arXiv:2402.05598](#).
- [63] X. Han, F. Hou, and H. Qin. UGrid: An Efficient-and-Rigorous Neural Multigrid Solver for Linear PDEs. 2024. [arXiv:2408.04846](#).
- [64] D. Greenfeld, M. Galun, R. Basri, I. Yavneh, and R. Kimmel. Learning to Optimize Multigrid PDE Solvers. *ICML*, 2019. [PMLR 97](#).
- [65] H. W. Engl, M. Hanke, and A. Neubauer. *Regularization of Inverse Problems*. Kluwer, 1996.

- [66] A. M. Stuart. Inverse Problems: A Bayesian Perspective. *Acta Numerica*, 2010.
- [67] B. T. Polyak. Some Methods of Speeding Up the Convergence of Iteration Methods. *USSR Computational Mathematics and Mathematical Physics*, 1964.
- [68] Y. Saad. *Iterative Methods for Sparse Linear Systems*, second edition. SIAM, 2003.
- [69] J. Barbier and N. Macris. The Adaptive Interpolation Method: A Simple Scheme to Prove Replica Formulas in Bayesian Inference. *Probability Theory and Related Fields*, 2019.
- [70] M. Bayati and A. Montanari. The Dynamics of Message Passing on Dense Graphs, with Applications to Compressed Sensing. *IEEE Transactions on Information Theory*, 2011.